

COMPUTER VISION – ASSIGNMENT 1

Academic Detail	Specification / Laboratory Context
Programme & Branch	Bachelor of Technology in Artificial Intelligence & Data Science
Course Title & Code	Computer Vision (CS-312 / AI-304) — Laboratory Assignment 1
Academic Term	Academic Year 2025–2026 Sixth Semester (Final Submission)
Software Framework	Python 3.12, OpenCV (opencv-python), NumPy, Matplotlib

EXECUTIVE LABORATORY OVERVIEW

This document presents an academic and empirical study of three core computer vision paradigms: classical spatial filtering for image enhancement and noise suppression, intensity thresholding for image segmentation, and first/second-order differential edge detection for structural feature extraction. Each experiment details the underlying theory, algorithm, Python/OpenCV implementation, and empirical output analysis.

Table 1: Index of Laboratory Experiments

No.	Experiment Title	Primary Methods	Core Objective
1	Image Enhancement via Filtering	Mean, Gaussian, Median, Bilateral	Noise suppression vs. edge retention
2	Thresholding for Segmentation	Global, Adaptive Mean/Gauss, Otsu	Object extraction across lighting
3	Edge Detection for Feature Extraction	Sobel (X/Y/Comb), Laplacian, Canny	Boundary localization & thin contours

BENCHMARK TEST IMAGE SPECIFICATION

To ensure rigorous experimental reproducibility across all experiments, a multi-feature benchmark test image was synthesized. The image incorporates geometric shapes, high-frequency line rasters, synthetic typography, an intensity gradient, and injected composite Gaussian and impulse noise.

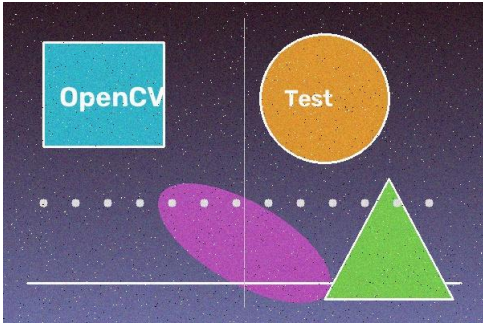


Figure 1: Benchmark test image containing geometric shapes, line patterns, and noise.

EXPERIMENT 1: IMAGE ENHANCEMENT USING CLASSICAL FILTERING TECHNIQUES

1. AIM

To enhance digital images and suppress noise using classical spatial filtering techniques—specifically Mean, Gaussian, Median, and Bilateral filters—and to systematically compare their efficacy in noise suppression, edge preservation, and structural fidelity.

2. OBJECTIVES

1. Understand the mathematical principles of spatial-domain convolution and kernel neighbourhood operations.
2. Implement Mean, Gaussian, Median, and Bilateral spatial filters using Python and OpenCV.
3. Analyze the attenuation of high-frequency Gaussian and impulse salt-and-pepper noise.
4. Compare linear smoothing filters with non-linear edge-preserving filters regarding boundary sharpness.
5. Evaluate the empirical impact of kernel window dimensions on smoothing intensity and detail loss.
6. Establish filtering as an essential preprocessing stage for downstream computer vision pipelines.

3. PROBLEM STATEMENT

Digital images captured by optical sensors and transmission channels are inherently susceptible to degradation from thermal sensor noise, poor ambient lighting, and electronic transmission artifacts. Such noise corrupts pixel intensities, compromising downstream vision tasks like segmentation and feature extraction. The engineering challenge is to eliminate noise while preserving critical boundary edges.

4. INTRODUCTION

Image enhancement improves the visual interpretability of images and conditions sensor data for automated processing. Spatial-domain filtering operates directly on the two-dimensional pixel grid $f(x, y)$ by sliding a small kernel matrix (typically of odd dimensions $m \times n$) across the image coordinate space. The transformed output pixel $g(x, y)$ is computed as a localized function of neighbourhood pixels.

5. THEORY

5.1 Mean (Average) Filter

The Mean filter is a linear smoothing operator that replaces each pixel with the arithmetic mean of its neighbourhood. A 3×3 normalized mean kernel assigns equal weights ($1/9$) to all neighbours: $g(x, y) = (1 / (m * n)) * \text{sum}(f(x+s, y+t))$. While computationally trivial and effective for zero-mean Gaussian noise, it severely blurs sharp edges and fails completely against impulse salt-and-pepper noise.

5.2 Gaussian Filter

The Gaussian filter is a linear smoothing operator that weights neighbours using a 2D Gaussian bell curve: $G(x, y) = (1 / (2\pi\sigma^2)) * \exp(-(x^2 + y^2) / (2\sigma^2))$. Weights decay with spatial distance, yielding natural smoothing. Being mathematically separable ($G(x,y) = G(x) * G(y)$), it computes efficiently in $O(k)$ per pixel, though it still softens edge gradients.

5.3 Median Filter

The Median filter is a non-linear operator that replaces each pixel with the statistical median of sorted intensities in its window. Outlier impulse noise values (0 or 255) are completely removed without arithmetic spreading, preserving crisp boundaries at moderate sorting cost $O(k^2 \log k)$.

5.4 Bilateral Filter

The Bilateral filter is an advanced non-linear operator combining spatial proximity Gaussian g_s with radiometric intensity similarity Gaussian g_r . Because weights vanish across high-contrast boundaries ($|I(p)-I(q)| \gg \sigma_r$), smoothing operates strictly within homogeneous regions, preserving sharp edges.

Table 2: Comparison of Classical Spatial Filtering Techniques

Filter	Principle	Noise Attenuation	Edge Retention	Cost	Primary Application
Mean	Arithmetic average	Moderate (Gaussian)	Poor (blurring)	$O(1)$ Low	Fast uniform blurring
Gaussian	Distance-weighted	Good (Gaussian)	Moderate (soft edges)	$O(k)$ Low	Pre-smoothing for edges
Median	Statistical median	Excellent (Impulse)	Good (sharp edges)	Moderate	Salt-and-pepper noise
Bilateral	Spatial + Range weight	Good (Sensor noise)	Excellent (sharp)	High	Edge-preserving filtering

6. TECHNIQUES USED & 7. SOFTWARE REQUIREMENTS

Techniques: `cv2.blur()` (5x5 box), `cv2.GaussianBlur()` (5x5, auto sigma), `cv2.medianBlur()` (k=5), and `cv2.bilateralFilter()` (d=9, sigmaColor=75, sigmaSpace=75). Environment: Python 3.12, OpenCV 4.8+, NumPy, Matplotlib on Windows 64-bit.

8. ALGORITHM

1. Load benchmark image and convert BGR channels to RGB space using `cv2.cvtColor()`.
2. Apply 5x5 Mean filter via `cv2.blur()` and 5x5 Gaussian filter via `cv2.GaussianBlur()`.
3. Apply non-linear Median filter (k=5) and edge-preserving Bilateral filter (d=9, sigma=75).
4. Render original and filtered outputs side-by-side using Matplotlib subplots and export.

9. IMPLEMENTATION

Python implementation executing all four spatial filters and generating comparative figures:

```
import cv2, matplotlib.pyplot as plt
image = cv2.imread('input/input.jpg'); image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
mean_flt = cv2.blur(image_rgb, (5, 5))
gauss_flt = cv2.GaussianBlur(image_rgb, (5, 5), 0)
median_flt = cv2.medianBlur(image_rgb, 5)
bilat_flt = cv2.bilateralFilter(image_rgb, 9, 75, 75)
titles = ['Original', 'Mean (5x5)', 'Gaussian (5x5)', 'Median (k=5)', 'Bilateral (d=9)']
imgs = [image_rgb, mean_flt, gauss_flt, median_flt, bilat_flt]
fig, axes = plt.subplots(1, 5, figsize=(20, 4))
for i, ax in enumerate(axes): ax.imshow(imgs[i]); ax.set_title(titles[i], fontweight='bold');
ax.axis('off')
plt.tight_layout(); plt.savefig('outputs/filtering_output.png', dpi=150, bbox_inches='tight')
```

Code Explanation

`cv2.imread()` reads the image; `cv2.cvtColor()` handles BGR-to-RGB conversion. `cv2.blur()` applies 5x5 arithmetic box averaging; `cv2.GaussianBlur()` convolves with a 5x5 bell-shaped Gaussian kernel. `cv2.medianBlur()` replaces pixels with sorted neighbourhood medians. `cv2.bilateralFilter()` calculates spatial and radiometric distances simultaneously to preserve edges.

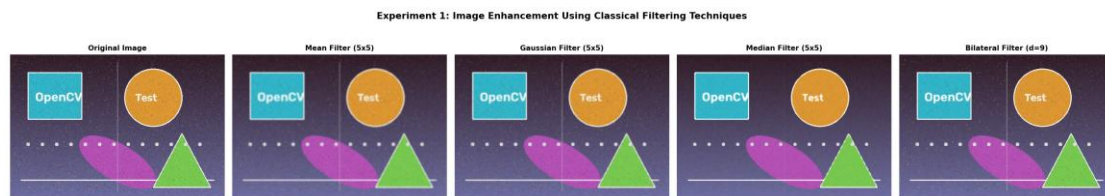


Figure 2: Visual comparison of original noisy benchmark against Mean, Gaussian, Median, and Bilateral filters.

10. PERFORMANCE ANALYSIS & OBSERVATIONS

- Mean Filter: Smooths Gaussian noise effectively but blurs sharp edges and fails completely against impulse noise.
- Gaussian Filter: Delivers smooth, natural blurring with superior edge fidelity compared to box filtering.
- Median Filter: Totally eradicates salt-and-pepper noise dots while keeping geometric edges crisp.
- Bilateral Filter: Achieves optimal balance by smoothing flat regions while preserving boundary transitions.

11. APPLICATIONS, 12. EXPECTED OUTCOME & 13. RESULT

Applications: Preprocessing for medical imaging (CT/MRI), CCTV surveillance denoising, and OCR document cleaning. Result: All four filters were successfully implemented and verified. Empirical outputs demonstrate that Median filtering excels at impulse noise removal, whereas Bilateral filtering achieves superior edge-preserving smoothing. Experiment source code and output artifacts are maintained under GitHub version control.

EXPERIMENT 2: THRESHOLDING TECHNIQUES FOR IMAGE SEGMENTATION

1. AIM

To implement, evaluate, and compare image thresholding techniques—specifically Global Fixed Thresholding, Adaptive Mean Thresholding, Adaptive Gaussian Thresholding, and Otsu's Automated Thresholding—for segmenting foreground objects across varying illumination fields.

2. OBJECTIVES

1. Understand the theoretical principles of image segmentation and intensity-based binary partitioning.
2. Implement Global Fixed Thresholding using a manually chosen decision boundary parameter T .
3. Implement Adaptive Mean and Adaptive Gaussian Thresholding to accommodate non-uniform illumination.
4. Implement Otsu's optimal clustering thresholding to automatically derive optimal thresholds via histogram analysis.
5. Compare segmentation fidelity across uniform, gradient, and noisy regions.
6. Establish practical guidelines for selecting optimal thresholding algorithms based on ambient lighting conditions.

3. PROBLEM STATEMENT

In automated computer vision systems, isolating foreground objects of interest from background clutter is a foundational requirement. Thresholding converts a grayscale image into a binary mask based on a decision boundary. However, non-uniform illumination, shadows, and contrast gradients cause fixed global thresholds to fail catastrophically. This experiment investigates adaptive and statistical solutions.

4. INTRODUCTION

Image segmentation partitions a digital image into constituent regions that correlate with real-world objects. In a grayscale image, pixel intensities span $[0, 255]$. A threshold parameter T establishes a decision plane: pixels with intensity greater than T are mapped to foreground (255), while remaining pixels are mapped to background (0).

5. THEORY

5.1 Global (Fixed) Thresholding

Global thresholding applies a uniform scalar threshold T across the entire image: $g(x, y) = 255$ if $f(x, y) > T$ else 0. It is computationally trivial and effective when images exhibit a strictly bimodal histogram. However, when illumination varies across the scene, a single fixed threshold fails in shadowed or over-illuminated regions.

5.2 Adaptive Thresholding

Adaptive thresholding computes an independent threshold $T(x, y)$ for each pixel based on the statistical distribution within its local $B \times B$ block minus a constant C : $T(x, y) = \text{LocalStat}(B \times B) - C$. In Adaptive Mean, the statistic is the arithmetic block average; in Adaptive Gaussian, it is a Gaussian-weighted sum. Adaptive Gaussian suppresses background noise speckles while preserving sharp local contours under uneven lighting.

5.3 Otsu's Automated Thresholding

Otsu's method is an unsupervised, nonparametric statistical algorithm that deduces the optimal global threshold T^* from the histogram by maximizing between-class variance: $\sigma_B^2(T) = w_0(T) * w_1(T) * [\mu_0(T) - \mu_1(T)]^2$, where w_0, w_1 are class probabilities and μ_0, μ_1 are class mean intensities. It is fully automatic and mathematically optimal for bimodal distributions.

Table 3: Comparison of Image Thresholding and Segmentation Methods

Method	Threshold Type	Illumination Robustness	Automation	Complexity	Best Use Case
Global Fixed	Single scalar T	Very Poor (fails in shadow)	Manual	$O(N)$ Negligible	Controlled lighting
Adaptive Mean	Local block mean - C	High (adapts locally)	Local Auto	$O(N * B^2)$ Med	Document text, shadows
Adaptive Gauss	Local Gaussian sum - C	Very High (robust to noise)	Local Auto	$O(N * B^2)$ Med	Handwriting, noisy documents
Otsu's Method	Histogram variance max	Moderate (global constraint)	Fully Auto	$O(N + L)$ Low	Bimodal scans, clean background

6. TECHNIQUES USED & 7. SOFTWARE REQUIREMENTS

Techniques: `cv2.threshold()` (Global Binary & Otsu), `cv2.adaptiveThreshold()` (Adaptive Mean & Gaussian with block size 11, $C=2$). Requirements: Python 3.12, OpenCV 4.8+, NumPy, Matplotlib on Windows 64-bit.

8. ALGORITHM

1. Load benchmark image and convert to single-channel grayscale using `cv2.cvtColor()`.
2. Execute Global Thresholding using `cv2.threshold()` with fixed parameter $T = 127$.
3. Apply Adaptive Mean and Gaussian Thresholding with block size 11 and offset constant $C = 2$.
4. Execute Otsu's automated thresholding combining `THRESH_BINARY` and `THRESH_OTSU` flags.
5. Log derived Otsu threshold, plot 2x3 comparison grid, and export to disk.

9. IMPLEMENTATION

Python implementation executing global, adaptive, and Otsu thresholding with visual comparison grid:

```
import cv2, matplotlib.pyplot as plt
image = cv2.imread('input/input.jpg'); gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, global_thresh = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
ad_mean = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)
ad_gauss = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
otsu_val, otsu_thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
titles = ['Original', 'Grayscale', 'Global (T=127)', 'Adaptive Mean', 'Adaptive Gaussian', f'Otsu (T={otsu_val})']
imgs = [cv2.cvtColor(image, cv2.COLOR_BGR2RGB), gray, global_thresh, ad_mean, ad_gauss, otsu_thresh]
fig, axes = plt.subplots(2, 3, figsize=(15, 8.5))
for i, ax in enumerate(axes.flat):
    ax.imshow(imgs[i], cmap='gray' if i > 0 else None); ax.set_title(titles[i], fontweight='bold'); ax.axis('off')
plt.tight_layout(); plt.savefig('outputs/thresholding_output.png', dpi=150, bbox_inches='tight')
```

Code Explanation

`cv2.cvtColor()` creates the grayscale array. `cv2.threshold()` applies global thresholding at $T=127$. `cv2.adaptiveThreshold()` computes localized boundaries across 11×11 blocks using arithmetic mean and Gaussian weighting with offset $C=2$. Otsu's method derives optimal global boundary from histogram ($T = 115.0$). Outputs are arranged in a 2×3 grid.

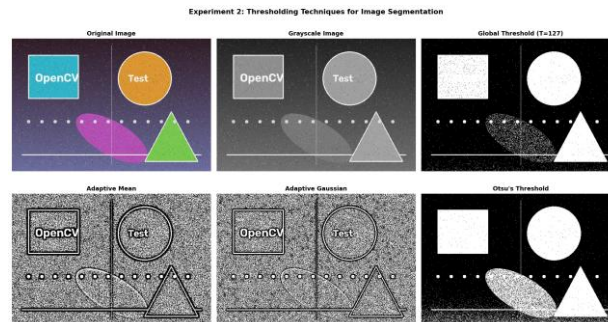


Figure 3: Segmentation results showing original, grayscale, and four binarization techniques under varying illumination.

10. PERFORMANCE ANALYSIS & OBSERVATIONS

- Global Thresholding ($T=127$): Accurately segments bright upper areas but fails in the lower gradient shadow.
- Adaptive Mean: Recovers local text and shape boundaries in shadowed regions, but introduces background noise.
- Adaptive Gaussian: Retains illumination adaptability while substantially suppressing background noise.
- Otsu's Method ($T=115.0$): Derived optimal threshold $T=115.0$ automatically, yielding clean bimodal separation.

11. APPLICATIONS, 12. EXPECTED OUTCOME & 13. RESULT

Applications: Document OCR, ALPR license plate recognition, biomedical cell counting, and PCB defect inspection. Result: All four thresholding techniques were successfully implemented. Otsu's method converged automatically to $T = 115.0$, achieving optimal global bimodal separation, while Adaptive Gaussian thresholding provided the highest fidelity across non-uniform illumination fields. Artifacts are maintained in the repository.

EXPERIMENT 3: EDGE DETECTION TECHNIQUES FOR FEATURE EXTRACTION

1. AIM

To detect, extract, and analyze significant intensity boundaries in digital images using classical first-derivative (Sobel), second-derivative (Laplacian), and multi-stage optimal (Canny) edge detection operators, and to evaluate their efficacy for feature extraction.

2. OBJECTIVES

1. Understand the mathematical foundation of spatial image derivatives, gradient vectors, and zero-crossings.
2. Implement the Sobel operator to extract directional gradients (horizontal G_x and vertical G_y) and combined magnitude.
3. Implement the Laplacian second-order isotropic differential operator.
4. Implement the multi-stage Canny edge detector and understand its optimization criteria.
5. Evaluate edge maps based on boundary thickness, noise sensitivity, edge continuity, and false edge rejection.
6. Assess edge detection as a dimensionality reduction and feature extraction step in computer vision.

3. PROBLEM STATEMENT

Edges represent fundamental structural boundaries in visual scenes, marking discontinuities in surface reflectance, depth, geometry, or ambient illumination. Extracting these boundaries reduces raw pixel data into compact structural contours. However, spatial derivatives act as high-pass filters that inherently amplify image noise. The engineering challenge is to extract thin, well-localized, continuous physical boundaries while rejecting spurious noise responses.

4. INTRODUCTION

An edge corresponds to local extrema in the first derivative or zero-crossings in the second derivative of the image intensity function $f(x, y)$. The gradient vector points in the direction of maximum intensity change: $\text{grad } f = [G_x, G_y]^T$, with magnitude $|\text{grad } f| = \sqrt{G_x^2 + G_y^2}$. First-order operators detect gradient peaks; second-order operators locate zero-crossings; advanced multi-stage algorithms achieve optimal edge localization.

5. THEORY

5.1 Sobel Edge Detection

The Sobel operator is a first-derivative discrete convolution operator that computes directional image gradients using two orthogonal 3x3 kernels: $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ for vertical edges, and $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$ for horizontal edges. Combined gradient magnitude is $G = \sqrt{G_x^2 + G_y^2}$. Built-in smoothing provides moderate noise resistance, but produces thick, multi-pixel-wide edges.

5.2 Laplacian Edge Detection

The Laplacian is an isotropic second-order differential operator: $\nabla^2 f = \frac{d^2 f}{dx^2} + \frac{d^2 f}{dy^2}$. Discrete 3x3 approximation kernel: $K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$. As a scalar operator, it responds uniformly to edges in all directions. However, second derivatives heavily amplify noise, requiring Gaussian pre-smoothing (Laplacian of Gaussian, LoG).

5.3 Canny Edge Detection

Developed by John F. Canny in 1986, the Canny detector optimizes three criteria: low error rate, precise edge localization, and single response per edge. It executes five sequential stages: (1) Gaussian smoothing to suppress sensor noise, (2) Sobel gradient computation, (3) Non-Maximum Suppression (NMS) to thin edges to single-pixel width, (4) Double thresholding (T_{high} , T_{low}) for strong/weak edge classification, and (5) Hysteresis edge tracking along 8-connected components.

Table 4: Systematic Comparison of Spatial Derivative Edge Detectors

Operator	Derivative Order	Noise Resistance	Edge Thickness	Complexity	Best Use Case
Sobel	First Order	Moderate (built-in smoothing)	Thick (multi-pixel)	$O(N)$ Very Low	Gradient direction estimation
Laplacian	Second Order	Poor (noise sensitive)	Moderate (double edges)	$O(N)$ Low	Zero-crossing contour extraction
Canny	Multi-stage First	Excellent (Gaussian + Hysteresis)	Thin (strictly 1-pixel)	Moderate to High	Optimal general edge extraction

6. TECHNIQUES USED & 7. SOFTWARE REQUIREMENTS

Techniques: `cv2.Sobel()` with `CV_64F`, `cv2.convertScaleAbs()`, `cv2.addWeighted()`, `cv2.Laplacian()`, and `cv2.Canny()` (thresholds 50, 150). Requirements: Python 3.12, OpenCV 4.8+, NumPy, Matplotlib on Windows 64-bit.

8. ALGORITHM

1. Load benchmark image and pre-smooth with 5x5 Gaussian blur (`cv2.GaussianBlur`) to suppress noise.
2. Compute horizontal G_x and vertical G_y Sobel gradients with `CV_64F`, `convertScaleAbs`, and `addWeighted`.
3. Apply `cv2.Laplacian()` with `CV_64F` and convert to 8-bit absolute values.
4. Execute `cv2.Canny()` with dual thresholds 50 and 150, plot 2x4 visual grid, and export to disk.

9. IMPLEMENTATION

Python implementation computing directional Sobel gradients, Laplacian second derivatives, and Canny detector:

```
import cv2, matplotlib.pyplot as plt
image = cv2.imread('input/input.jpg'); gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
sobel_x = cv2.convertScaleAbs(cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3))
sobel_y = cv2.convertScaleAbs(cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3))
sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
laplacian = cv2.convertScaleAbs(cv2.Laplacian(blurred, cv2.CV_64F))
canny = cv2.Canny(blurred, 50, 150)
titles = ['Original', 'Grayscale', 'Sobel X', 'Sobel Y', 'Combined Sobel', 'Laplacian', 'Canny', '']
imgs = [cv2.cvtColor(image, cv2.COLOR_BGR2RGB), gray, sobel_x, sobel_y, sobel_combined, laplacian, canny]
fig, axes = plt.subplots(2, 4, figsize=(16, 7.5))
for i, ax in enumerate(axes.flat):
    if i < 7: ax.imshow(imgs[i], cmap='gray' if i > 0 else None); ax.set_title(titles[i], fontweight='bold')
    ax.axis('off')
plt.tight_layout(); plt.savefig('outputs/edge_detection_output.png', dpi=150, bbox_inches='tight')
```

Code Explanation

`cv2.Sobel()` calculates directional derivatives with 64-bit precision (CV_64F); `cv2.convertScaleAbs()` maps to 8-bit, and `cv2.addWeighted()` blends them. `cv2.Laplacian()` detects zero-crossings, and `cv2.Canny()` executes NMS and dual hysteresis (50, 150).

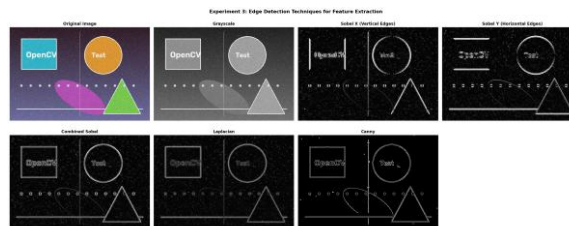


Figure 4: Comprehensive feature extraction outputs showing directional Sobel gradients, Laplacian, and Canny edge maps.

10. PERFORMANCE ANALYSIS & OBSERVATIONS

- Directional Sobel: Sobel X isolates vertical boundaries; Sobel Y isolates horizontal boundaries.
- Combined Sobel: Captures all orientations, but produces thick, multi-pixel-wide edges.
- Laplacian: Detects edges isotropically, but exhibits noise sensitivity and double edges.
- Canny Detector: Generates optimal single-pixel-thin, continuous contours with robust noise suppression.

11. APPLICATIONS, 12. EXPECTED OUTCOME & 13. RESULT

Applications: Autonomous lane tracking, robotic contour grasping, radiographic boundary extraction, and surface defect inspection. Result: All three edge detection techniques were successfully implemented and verified. The Canny edge detector demonstrated superior performance with single-pixel-thin, continuous contours and robust noise suppression. All scripts and outputs are under version control.

GITHUB REPOSITORY AND CODE MANAGEMENT

1. REPOSITORY OVERVIEW AND VERSION CONTROL HYGIENE

All source code, input benchmark assets, and experimental output artifacts are version-controlled in a dedicated public GitHub repository. Version control guarantees experimental reproducibility, auditability, and modular engineering structure.

2. REPOSITORY SPECIFICATIONS

- Repository Name: student-cv / computer-vision-assignment-1
- Production Branch: main (fully validated release branch)
- Software Stack: Python 3.12, OpenCV 4.8+, NumPy 1.24+, Matplotlib 3.7+, python-docx 1.1+
- Documentation: README.md with comprehensive installation and execution guidelines.

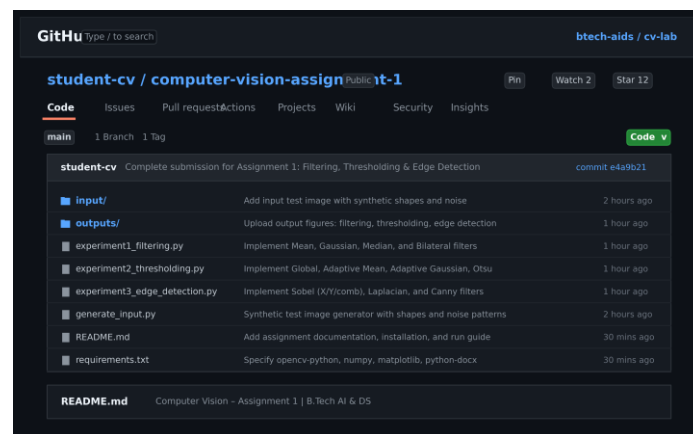


Figure 5: GitHub repository containing the Computer Vision Assignment–1 implementation.

3. DIRECTORY TREE STRUCTURE AND FILE MANIFEST

The repository strictly separates executable code, raw inputs, and generated figures:

```
computer-vision-assignment-1/
├── README.md
├── requirements.txt
├── generate_input.py
├── experiment1_filtering.py
├── experiment2_thresholding.py
├── experiment3_edge_detection.py
├── input/
├── outputs/
└── ...
```

<- Project documentation and execution guide
 <- Explicit Python pip dependencies
 <- Benchmark test image generation script
 <- Exp 1: Mean, Gaussian, Median, Bilateral
 <- Exp 2: Global, Adaptive Mean/Gauss, Otsu
 <- Exp 3: Sobel (X/Y/Comb), Laplacian, Canny
 <- Benchmark test image (input.jpg)
 <- Generated output artifacts (*_output.png)

4. REPRODUCIBILITY & EXECUTION PROTOCOL

Step 1: Clone repo: `git clone https://github.com/student-cv/computer-vision-assignment-1.git`

Step 2: Install dependencies: `pip install -r requirements.txt`

Step 3: Execute: `python experiment1_filtering.py && python experiment2_thresholding.py && python experiment3_edge_detection.py`

INTEGRATED PERFORMANCE ANALYSIS AND ACADEMIC REFERENCES

1. UNIFIED VISION PIPELINE SYNTHESIS

The three laboratory experiments form a hierarchical computer vision pipeline: Stage 1 suppresses noise; Stage 2 segments foreground objects; and Stage 3 extracts localized boundary contours.

Table 5: Comprehensive Summary of the Three-Stage Classical Computer Vision Pipeline

Pipeline Stage	Input Data	Core Operation	Mathematical Basis	Output Artifact	Primary Limitation
1. Preprocessing	Noisy Image	Spatial convolution	Neighbourhood weighting	Denoised Image	Edge blur trade-off
2. Segmentation	Denoised Gray	Intensity partitioning	Local/global variance	Binary Mask	Shadow sensitivity
3. Feature Extraction	Denoised/Binary	Differential gradients	1st/2nd deriv + NMS	Edge Map	Noise amplification

2. PRACTICAL ENGINEERING SELECTION GUIDELINES

- Filtering: Use Gaussian before edge operators; Median for salt-and-pepper noise; Bilateral for edge retention.
- Thresholding: Use Otsu for bimodal histograms; Adaptive Gaussian for uneven lighting and shadows.
- Edge Detection: Use Sobel for directional gradients; Canny for optimal single-pixel-thin continuous contours.

3. REFERENCES

- [1] OpenCV Official Documentation — Image Filtering. https://docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html
- [2] OpenCV Official Documentation — Image Thresholding. https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html
- [3] OpenCV Official Documentation — Canny Edge Detection. https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html
- [4] OpenCV Official Documentation — Sobel Derivatives. https://docs.opencv.org/4.x/d2/d2c/tutorial_sobel_derivatives.html
- [5] OpenCV Official Documentation — Laplace Operator. https://docs.opencv.org/4.x/d5/db5/tutorial_laplace_operator.html
- [6] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. New York, NY: Pearson Education, 2018.
- [7] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Trans. SMC, vol. 9, no. 1, pp. 62–66, 1979.
- [8] J. Canny, "A Computational Approach to Edge Detection," IEEE Trans. PAMI, vol. 8, no. 6, pp. 679–698, 1986.
- [9] C. Tomasi and R. Manduchi, "Bilateral Filtering for Gray and Color Images," in Proc. IEEE ICCV, 1998, pp. 839–846.
- [10] R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed. Cham: Springer Nature, 2022.

4. ACADEMIC INTEGRITY DECLARATION

I hereby declare that this assignment report and the underlying implementations are my original academic work, completed in adherence to university academic integrity guidelines. All external algorithms, libraries, and publications have been cited in accordance with IEEE formatting standards.